University Library System

Midterm Project Report | Kuanysh Asker, Zhan Sharkhanov, Zhanibek Uzakbai

Project overview

We made a console-based university library program in Python using SQLite. It stores books, members, and borrowing records. The starting database has 50 books, 40 members, and 30 borrowings, meeting the library dataset minimums.

Team work

Team member	Tasks
Kuanysh Asker	1. Add information 2. Display records
Zhan Sharkhanov	3. Calculate results and statistics 4. Search for information
Zhanibek Uzakbai	5. Update and delete information 6. Save and export results

Database tables

Table	Information stored
books	Book ID, title, author, genre, and availability
members	Member ID, name, email, and study program
borrowings	Borrowing ID, book ID, member ID, and returned status

Classes and main functions

The Book class stores a book ID, title, author, genre, and availability; the Member class stores a member ID, name, email, and study program. Their show() methods print each record. Main functions include create_tables, add_book, add_member, display_records, search_records, show_statistics, update_book, delete_record, return_book, and export_results. show_menu and main run the menu loop.

The program uses regular expressions (re) to check email format and random to assign sample borrowing records to members. It also validates required fields and IDs. The borrowings table links books and members by their IDs.

The menu keeps running until the user chooses Exit. Each borrowing record refers to a book and a member, so the program can show loan status and update availability when a book is returned.

Adding and displaying records

My part was adding books and members and displaying the stored records. The add functions check required fields, validate member email, and save records to SQLite. The display functions read the selected table and print its rows.

```
133
134 def add_book(connection):
135     title = input("Book title: ")
136     author = input("Author: ")
137     genre = input("Genre: ")
138     if title == "" or author == "" or genre == "":
139         print("Please fill in all book details.")
140         return
141     connection.execute("INSERT INTO books (title, author, genre, available) VALUES (?, ?, ?, 1)", (title, author, genre))
142     connection.commit()
143     print("Book added.")
144
145
146 def add_member(connection):
147     name = input("Member name: ")
148     email = input("Member email: ")
149     program = input("Study program: ")
150     if name == "" or program == "" or re.match(r"^[\w.-]+@[\w.-]+\.\w+$", email) is None:
151         print("Enter a name, study program, and valid email.")
152         return
153     existing = connection.execute("SELECT member_id FROM members WHERE email=?", (email,)).fetchone()
154     if existing is not None:
155         print("That email is already registered.")
156         return
157     connection.execute("INSERT INTO members (name, email, program) VALUES (?, ?, ?)", (name, email, program))
158     connection.commit()
159     print("Member added.")
160
```

Figure 1. Code for adding books and members.

```
187
188 def display_books(connection):
189     print("\nBOOKS")
190     rows = connection.execute("SELECT book_id, title, author, genre, available FROM books").fetchall()
191     show_book_table(rows)
192
193
194 def display_members(connection):
195     print("\nMEMBERS")
196     rows = connection.execute("SELECT member_id, name, email, program FROM members").fetchall()
197     show_member_table(rows)
198
199
200 def display_borrowings(connection):
201     print("\nBORROWINGS")
202     print("{:<4} | {:<9} | {:<10} | {:<10}".format(
203         "ID", "BOOK ID", "MEMBER ID", "STATUS"
204     ))
205     print("-" * 43)
206     rows = connection.execute("SELECT borrowing_id, book_id, member_id, returned FROM borrowings").fetchall()
207     for row in rows:
208         status = "Returned" if row[3] == 1 else "Borrowed"
209         print("{:<4} | {:<9} | {:<10} | {:<10}".format(row[0], row[1], row[2], status))
210     if len(rows) == 0:
211         print("No borrowing records found.")
212
213
214 def display_records(connection):
215     print("1. Books")
216     print("2. Members")
217     print("3. Borrowings")
218     choice = input("Choose records to display: ")
219     if choice == "1":
220         display_books(connection)
221     elif choice == "2":
222         display_members(connection)
223     elif choice == "3":
224         display_borrowings(connection)
225     else:
226         print("Invalid choice.")
227
```

Figure 2. Functions for displaying books, members, and borrowings.

```
162 def show_book_table(rows):
163     if len(rows) == 0:
164         print("No books found.")
165         return
166     print("{:<4} | {:<42} | {:<34} | {:<20} | {:<9}".format(
167         "ID", "TITLE", "AUTHOR", "GENRE", "AVAILABLE"
168     ))
169     print("-" * 122)
170     for row in rows:
171         book = Book(row[0], row[1], row[2], row[3], row[4])
172         book.show()
173
174
175 def show_member_table(rows):
176     if len(rows) == 0:
177         print("No members found.")
178         return
179     print("{:<4} | {:<25} | {:<42} | {:<22}".format(
180         "ID", "NAME", "EMAIL", "PROGRAM"
181     ))
182     print("-" * 105)
183     for row in rows:
184         member = Member(row[0], row[1], row[2], row[3])
185         member.show()
186
```

Figure 3. Helpers that format book and member rows.

Program results for add and display

The terminal confirms that a new book and member were added. The display option then lists book, member, and borrowing records with their fields in columns.

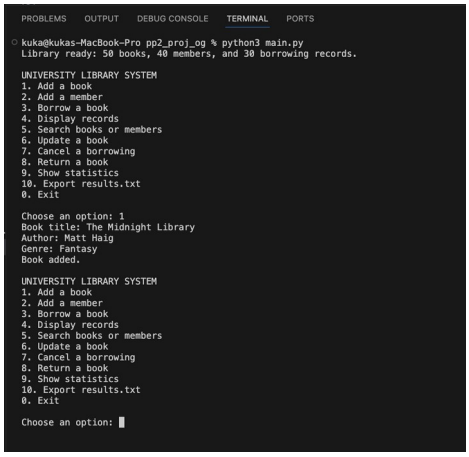


Figure 4. Adding a book.



Figure 5. Adding a member.

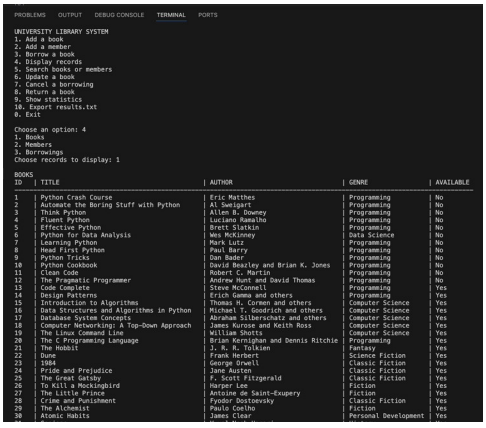


Figure 6. Displayed book records.

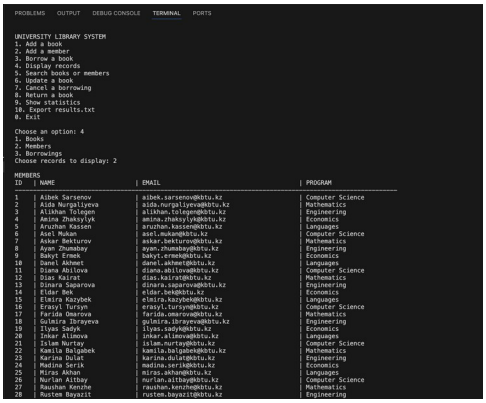


Figure 7. Displayed member records.

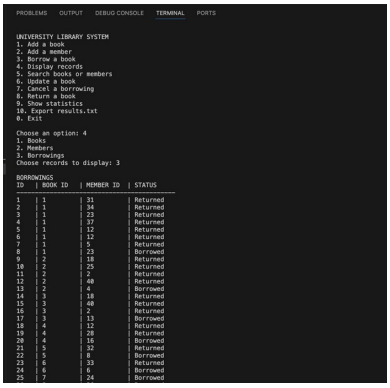


Figure 8. Displayed borrowing records and loan status.

Zhan: statistics and search

Zhan worked on statistics and search. The statistics function counts all borrowing records, counts books that are currently on loan, and groups borrowing records by book to find the most frequently borrowed title. These are the required calculations from the database.

```
313
314   def show_statistics(connection):
315       total = connection.execute("SELECT COUNT(*) FROM borrowings").fetchone()[0]
316       current = connection.execute("SELECT COUNT(*) FROM borrowings WHERE returned=0").fetchone()[0]
317       top_book = connection.execute("SELECT book_id, COUNT(*) FROM borrowings GROUP BY book_id ORDER BY COUNT(*) DESC LIMIT 1").fetchone()
318       print("Total borrowing records:", total)
319       print("Books currently borrowed:", current)
320       if top_book is not None:
321           title = connection.execute("SELECT title FROM books WHERE book_id=?", (top_book[0],)).fetchone()
322           print("Most frequently borrowed book:", title[0], "-", top_book[1], "loans")
323
```

Figure 9. Statistics code.

```
228
229   def search_records(connection):
230       print("1. Search book by title")
231       print("2. Search member by name or ID")
232       choice = input("Choose search: ")
233       if choice == "1":
234           title = input("Enter part of the title: ")
235           rows = connection.execute("SELECT book_id, title, author, genre, available FROM books WHERE title LIKE ?", ("%" + title + "%",)).fetchall()
236           print("\nSEARCH RESULTS: BOOKS")
237           show_book_table(rows)
238       elif choice == "2":
239           name = input("Enter a name or member ID: ")
240           rows = connection.execute("SELECT member_id, name, email, program FROM members WHERE name LIKE ? OR member_id=?", ("%" + name + "%", name)).fetchall()
241           print("\nSEARCH RESULTS: MEMBERS")
242           show_member_table(rows)
243       else:
244           print("Invalid choice.")
245
```

Figure 10. Search code.

Results from the program

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

kuka@kukas-MacBook-Pro pp2_proj_og % python3 main.py
Library ready: 50 books, 40 members, and 30 borrowing records.

UNIVERSITY LIBRARY SYSTEM
1. Add a book
2. Add a member
3. Borrow a book
4. Display records
5. Search books or members
6. Update a book
7. Cancel a borrowing
8. Return a book
9. Show statistics
10. Export results.txt
0. Exit

Choose an option: 9
Total borrowing records: 30
Books currently borrowed: 12
Most frequently borrowed book: Python Crash Course - 8 loans

UNIVERSITY LIBRARY SYSTEM
1. Add a book
2. Add a member
3. Borrow a book
4. Display records
5. Search books or members
6. Update a book
7. Cancel a borrowing
8. Return a book
9. Show statistics
10. Export results.txt
0. Exit

Choose an option:
```

Figure 11. Statistics: 30 records, 12 active loans, and Python Crash Course with 8 loans.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Library ready: 50 books, 40 members, and 30 borrowing records.

UNIVERSITY LIBRARY SYSTEM
1. Add a book
2. Add a member
3. Borrow a book
4. Display records
5. Search books or members
6. Update a book
7. Cancel a borrowing
8. Return a book
9. Show statistics
10. Export results.txt
0. Exit

Choose an option: 5
1. Search book by title
2. Search member by name or ID
Choose search: 2
Enter a name or member ID: Inkar

SEARCH RESULTS: MEMBERS
ID   | NAME                | EMAIL                          | PROGRAM
--------------------------------------------------------------------------------
20   | Inkar Alimova       | inkar.alimova@kbtu.kz          | Languages

UNIVERSITY LIBRARY SYSTEM
1. Add a book
2. Add a member
3. Borrow a book
4. Display records
5. Search books or members
6. Update a book
7. Cancel a borrowing
8. Return a book
9. Show statistics
10. Export results.txt
0. Exit

Choose an option:
```

Figure 12. Search for a member by part of a name.

The statistics use database records to count total and active loans and find the most borrowed title. Search can match part of a book title or member name, or a member ID; searching “Inkar” returns Inkar Alimova.

Zhanibek: updating, deleting, and exporting

Zhanibek's update_book function checks the book ID and new details before saving a change. His return function updates both the borrowing status and book availability. Cancelling a loan deletes its record and makes the book available again if it was still checked out.



Figure 13. Updating a book and the program response.

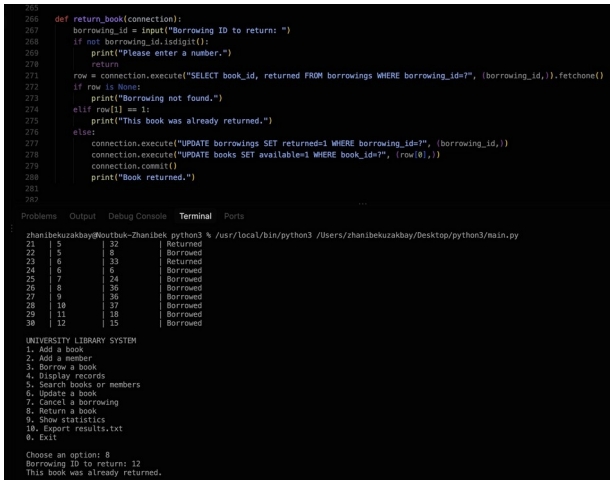


Figure 14. Returning a book.

The export_results function writes results.txt using database queries. It includes record totals, active loans, the most borrowed book, and the borrowing list. The latest screenshot shows 50 books, 40 members, 30 borrowings, 12 current loans, and Python Crash Course with 8 loans.

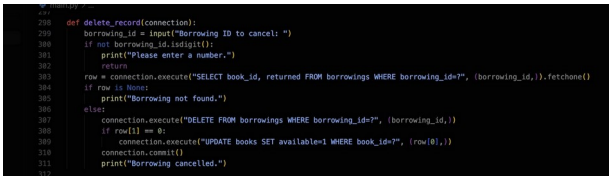


Figure 15. Code for cancelling a borrowing.

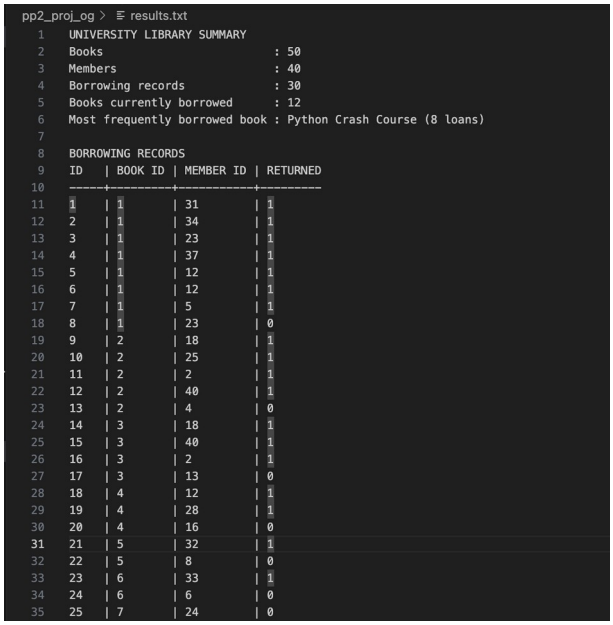


Figure 16. Current exported results.txt.

Conclusion

Our team combined all six tasks in one library program. I worked on adding and displaying records, Zhan on statistics and search, and Zhanibek on updates, cancellations, returns, and exporting. The project uses three related database tables and saves its summary to a text file.